

Sampling with `tfq.layers.Sample()` produces same outputs

<https://github.com/tensorflow/quantum/issues/550>

ROW 124

Sampling with tfq.layers.Sample() produces same outputs #550

[New issue](#)[Closed](#)

therooler opened on Apr 28, 2021

...

Python 3.7
Tensorflow 2.3.1
Tensorflow Quantum 0.4.0

Hi,

I ran into something strange when I was trying to calculate errorbars on the estimators of some observables. It seems that there are differences between how the `cirq.Simulator()` object handles random numbers compared to how this is done in the default `tfq.layers.Sample()` backend.

Expected behavior

Calling `tfq.layers.Sample()` twice on the same circuit within a script should give me different bitstrings. Launching the same script multiple times where I sample from a circuit should give me different bitstring for each script.

Actual behaviour

Multiple scripts executed at different times gave me exactly the same set bitstrings. Also, reinitializing the sampling layer within a script can give the same set of bitstrings depending on what backend one uses.

Here is a minimal working example illustrating the problem:

```
import tensorflow_quantum as tfq
import tensorflow as tf
import cirq
import numpy as np

CASE = 'tfq'

def sample_generator(circuits_tf, repetitions, nqubits):
    # take tfq backend or cirq backend
    if CASE == 'tfq':
        sample_layer = tfq.layers.Sample()
    elif CASE == 'cirq':
        sample_layer = tfq.layers.Sample(cirq.Simulator())
    # create generator from sample_layer outputs, repeat 'repetition' times.
    data_x = tf.reshape(sample_layer(circuits_tf, repetitions=repetitions).to_tensor(), (-1, nqubits))
    data_x = tf.unstack(data_x)
    for d in data_x:
        yield d

def main():
    # tf.random.set_seed('supcom')

    nqubits = 4
    nsamples = 100
    batch_size = 10
    # create a simple circuit
    qubits = cirq.GridQubit.rect(1, 4)
    circuit = cirq.Circuit()
    circuit.append(cirq.H.on_each(qubits))

    # convert to tfq tensor
    circuits_tf = tfq.convert_to_tensor([circuit])

    # create data generator
    sample_gen = tf.data.Dataset.from_generator(sample_generator, args=(circuits_tf, nsamples, nqubits),
                                              output_types=tf.int32)
    sample_gen = sample_gen.batch(batch_size)
    sample_gen = sample_gen.prefetch(3 * batch_size)

    # get samples and return them
    samples = []
    for batch in sample_gen:
        samples.append(batch.numpy())
    return samples

if __name__ == '__main__':
    samples_1 = main()
    samples_2 = main()
    for s1, s2 in zip(samples_1, samples_2):
        # print(s1, s2)
        print(np.allclose(s1, s2))
```

Assignees

MichaelBroughton

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

Customize

Subscribe

You're not receiving notifications from this thread.

Participants



Zoterc

I consider the following cases:

1.) CASE=='tfq'

Calling main() twice gives a different set of 100 bitstrings for samples_1 and samples_2 (only False is printed in the final line)

2.) CASE=='tfq' , two separate script executions.

Again, different bitstrings for samples_1 and samples_2 , but the two scripts give the same output:

Run 1

Run 2

Same Samples

3.) CASE=='cirq'

Calling main() twice gives exactly the same 100 bitstrings for samples_1 and samples_2 (only True is printed in the final line)

4.) CASE=='cirq'

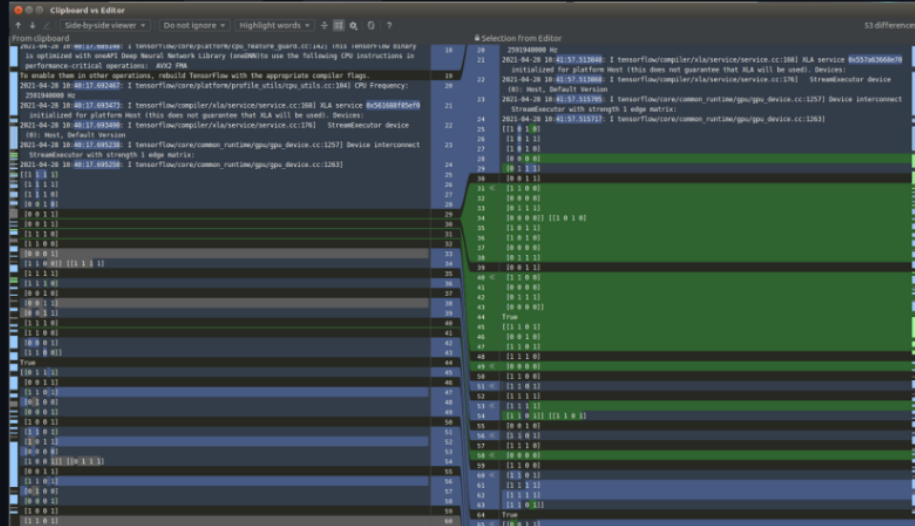
Again, the bitstrings for samples_1 and samples_2 are equal, but now the two scripts give different output

Run 1

Run 2

Same Samples

Again, the bitstrings for `samples_1` and `samples_2` are equal, but now the two scripts give different output



(All of this is unaffected by changing the random seed in Tensorflow between scripts.)

From this I deduce that in TFQ, initializing the Sampling layer initializes a global random number generator once with a fixed seed independent of time (at which the script is executed), and this random number generator is reused for all subsequent calls. This would explain 1.) and 2.).

Whereas with the Cirq backed, the random number generator is reinitialized when the Sampling layer is initialized, but the seed is now dependent on the time (at which the script is executed), which would explain 3.) and 4.).

I fixed my problems by passing a seed to the `cirq.Simulator()` object that depends on time

I this expected behavior with the way I coded this up? Is there a better way to sample bitstrings from the circuit and ensure that everytime I call the sampling layer I get different bitstrings?

Best,
Roeland



MichaelBroughton on Apr 28, 2021

Collaborator ...

This might have something to do with #534 . Would you be able to quickly run this same code using `tensorflow==2.4.1` and `tfq-nightly` and see if you are still getting this error ?



therooler on Apr 28, 2021

Contributor Author ...

With `tensorflow==2.4.1` and `tfq-nightly-0.5.0.dev20210428`, changing

```
sample_gen = tf.data.Dataset.from_generator(sample_generator, args=(circuits_tf, nsamples, nqubits),  
                                             output_types=tf.int32)
```

to

```
sample_gen = tf.data.Dataset.from_generator(sample_generator, args=(circuits_tf, nsamples, nqubits),  
                                             output_types=tf.int8)
```

runs without errors, and gives

- 1.) Same
- 2.) Same, but different scripts now give different bitstrings, yay!
- 3.) Same

G

- 1.) Same
- 2.) Same, but different scripts now give different bitstrings, yay!
- 3.) Same
- 4.) Same



MichaelBroughtton on Apr 28, 2021

Collaborator ...

Hmmmm that still seems like it might be worth investigating. I'm going to look into what we're doing with our randomness generation and see if we need to fix anything up some more.



MichaelBroughtton mentioned this on Apr 29, 2021

[Switch to using tf.philox.random. #551](#)



MichaelBroughtton self-assigned this on May 3, 2021



MichaelBroughtton on May 4, 2021

Collaborator ...

@therooler Could you try updating to the latest tf nightly (the one that includes [#551](#)) and give an update on the behavior you're seeing? I'm hoping that this has solved the final issue with the bit strings here.



therooler on May 4, 2021

Contributor Author ...

Using `tfq-nightly-0.5.0.dev20210504` I get the same behavior as in my previous comment.

But this is what we expect right?

- 1.) Within the same script, calling the sampler twice gives two chains of different samples.
- 2.) Executing the same script twice gives different samples.

So we never end up with the same bitstrings.



MichaelBroughtton on May 5, 2021

Collaborator ...

Yes it is what we expect, just wanted to make sure the last changes hadn't re-introduced any bad behavior. I think this is safe to close now.



1



MichaelBroughtton closed this as [completed](#) on May 5, 2021



jaeyoo added a commit that references this issue on Mar 30, 2023

Merge pull request [tensorflow#550](#) from quantumlib/release-v0_13_3

Verified 1b3d0f1

Switch to using tf philox random. #551

<> Code

Merged MichaelBroughton merged 3 commits into master from philox_random on May 3, 2021

Conversation 3 Commits 3 Checks 0 Files changed 8 +156 -88



MichaelBroughton commented on Apr 29, 2021 Collaborator

Upgrades or random number generation to TF built in random number generator. xref #550

Switch to using tf philox random. c50f1d7

MichaelBroughton requested review from zaqqwerty and jaeyoo 4 years ago

jaeyoo approved these changes on May 2, 2021 View reviewed changes



jaeyoo left a comment Member

LGTM with nits: why some uses Rand64 , but others use Rand32 ? Doesn't it require to be equal to Rand64 to cover up to the future qubit sizes?



MichaelBroughton commented on May 3, 2021 Collaborator Author

In the cases where Rand64 is used, qsim requires uint64 and in the case where Rand32 is used qsim only requires uint32. I have no idea why they need random seeds with 32 bits and 64 bits in other places, but I just made sure to follow the types.

jaeyoo approved these changes on May 3, 2021 View reviewed changes



jaeyoo left a comment Member

LGTM

Reviewers

 jaeyoo

 zaqqwerty

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Successfully merging this pull request may close these issues.

None yet

Notifications Customize

Subscribe

You're not receiving notifications from this thread.

2 participants

